

1 Problem statement

Given an $n \times n$ matrix a , we have to move from the upper-left cell $(1, 1)$ to the lower-right cell (n, n) . In one step we may go to any of the four orthogonal neighbours (up, down, left, right). While walking we may change direction at most **twice** – i.e. the walk may contain at most two bends (corners). All cells may be visited more than once, but the total sum of the visited cells (including the start and the finish) has to be maximised.

2 Key observation

A walk that contains at most two direction changes consists of at most three straight segments. Consequently, while we are walking we only have to know

- how many bends have already been used,
- and whether the last segment is horizontal or vertical.

Thus the whole walk can be described with three simple DP states.

3 Dynamic programming

For every cell (i, j) we keep four tables:

- $dp[i][j]$: maximum sum of a walk from $(1, 1)$ to (i, j) using at most two bends.
- $dp0[i][j]$: maximum sum of a walk that ends in (i, j) and the last step was made without a bend (i.e. we come from the cell above or from the left).
- $dp1[i][j]$: maximum sum of a walk that ends in (i, j) and exactly one bend has already been used.
- $dp2[i][j]$: maximum sum of a walk that ends in (i, j) and exactly two bends have already been used.

The recurrences are obtained by looking at the last step of a walk.

1. **No new bend.** When the last step arrives from above $(i - 1, j)$ or from the left $(i, j - 1)$ we do not introduce a bend, therefore we may take the best walk arriving at those neighbours without caring about the number of bends already used:

$$dp[i][j] = \max(dp[i - 1][j], dp[i][j - 1]) + a_{i,j}. \quad (1)$$

2. **First bend.** A walk that makes its *first* bend while entering (i, j) must come from the cell $(i - 1, j + 1)$, i.e. one step north-east of (i, j) . From that neighbour we have two possibilities:

- the walk already contained a bend – state $dp1[i - 1][j + 1]$;

- or it contained no bend – state $dp0[i-1][j+1]$, and the current step creates the first bend.

Hence

$$dp1[i][j] = \max(dp1[i-1][j+1], dp0[i-1][j+1]) + a_{i,j}. \quad (2)$$

3. **Second bend.** Analogously, a walk that makes its *second* bend while entering (i, j) must come from the cell $(i+1, j-1)$, i.e. one step south-west of (i, j) . The predecessor can be in state $dp2$ (already two bends) or in state $dp0$ (the step creates the second bend):

$$dp2[i][j] = \max(dp2[i+1][j-1], dp0[i+1][j-1]) + a_{i,j}. \quad (3)$$

4. **Result for the cell.** The best walk that ends in (i, j) uses at most two bends, therefore

$$dp[i][j] = \max(dp0[i][j], dp1[i][j], dp2[i][j]). \quad (4)$$

The start cell is initialised with $dp[1][1] = dp0[1][1] = dp1[1][1] = dp2[1][1] = a_{1,1}$.

4 Processing order

All formulas use only cells whose sum $i + j$ is strictly smaller (??) or equal (??-??). Therefore we iterate over antidiagonals $s = 3, \dots, 2n$ (the antidiagonal $s = 2$ consists only of the start cell).

For a fixed s the order is

1. compute all $dp0$ (depends on already known dp);
2. compute all $dp1$ – forward scan of the antidiagonal ($i = 1 \rightarrow n$);
3. compute all $dp2$ – backward scan of the antidiagonal ($i = n \rightarrow 1$);
4. finally compute dp .

Because $dp0[i-1][j+1]$ has already been computed in step1, the forward scan guarantees its availability; the backward scan guarantees the same for $dp0[i+1][j-1]$.

All unreachable states are initialised with a very negative number $-\text{INF}$ ($= -1\,000\,000\,001$), so they never become a maximum.

5 Correctness proof (sketch)

We prove by induction over the antidiagonal order that every table entry contains the maximum possible sum for the corresponding class of walks.

- **Base.** For cell $(1, 1)$ the only walk consists of this single cell, hence the initial assignment is optimal for all four tables.
- **Inductive step.** Assume the claim holds for all cells with $i + j < s$. Consider a cell (i, j) with $i + j = s$.
 - Any optimal walk that ends in (i, j) without a bend at the last step must come from $(i-1, j)$ or $(i, j-1)$. By the induction hypothesis the best sums arriving at those neighbours are $dp[i-1][j]$ and $dp[i][j-1]$, hence formula ?? yields the optimal value for $dp0[i][j]$.
 - Any optimal walk that ends in (i, j) with exactly one bend must have its previous cell on the same antidiagonal $(i-1, j+1)$. If the walk already had a bend, its value is $dp1[i-1][j+1]$ (optimal by induction); if it had none, its value is $dp0[i-1][j+1]$ and the current step creates the first bend. Taking the maximum of the two gives the optimal value for $dp1[i][j]$, which is exactly ??.
 - The argument for $dp2$ is symmetric, using the cell $(i+1, j-1)$.
 - Finally, a walk that uses at most two bends belongs to one of the three classes, consequently its sum is bounded by the right-hand side of ??. Conversely each of the three values is realised by a walk of the corresponding class, therefore $dp[i][j]$ equals the optimum.

Thus the claim holds for (i, j) , completing the induction.

- **Termination.** When $i = n$ and $j = n$ the table $dp[n][n]$ contains the maximum sum of any walk from $(1, 1)$ to (n, n) that changes direction at most twice.

6 Complexity analysis

The four $n \times n$ tables require $\Theta(n^2)$ memory. Every antidiagonal processes each cell once, therefore the total running time is also $\Theta(n^2)$. With $n \leq 1000$ this easily satisfies the limits.

7 Reference implementation

The following code follows exactly the algorithm proved correct above.

```
#include <bits/stdc++.h>
#define INF 1000000001
#define L 1005
```

```

using namespace std;
ifstream fin("birocratie.in");
ofstream fout("birocratie.out");

int n;
int m[L][L], dp[L][L], dp0[L][L], dp1[L][L], dp2[L][L];

void board() {
    for (int i = 0; i < n + 2; i++)
        for (int j = 0; j < n + 2; j++)
            m[i][j] = dp[i][j] = dp0[i][j] = dp1[i][j] = dp2[i][j]
                = -INF;
}

int main() {
    fin >> n;
    board();
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            fin >> m[i][j];

    dp[1][1] = dp0[1][1] = dp1[1][1] = dp2[1][1] = m[1][1];

    for (int antidiag = 3; antidiag <= 2 * n; antidiag++) {
        // step 1 : dp0 (no new bend)
        for (int i = 1; i <= n; i++) {
            int j = antidiag - i;
            if (j < 1 || j > n) continue;
            dp0[i][j] = max({dp[i - 1][j], dp[i][j - 1]}) + m[i][j];
        }

        // step 2 : dp1 (first bend)          forward scan
        for (int i = 1; i <= n; i++) {
            int j = antidiag - i;
            if (j < 1 || j > n) continue;
            dp1[i][j] = max({dp1[i - 1][j + 1], dp0[i - 1][j + 1]}) + m[i][j];
        }

        // step 3 : dp2 (second bend)         backward scan
        for (int i = n; i > 0; i--) {
            int j = antidiag - i;
            if (j < 1 || j > n) continue;
            dp2[i][j] = max({dp2[i + 1][j - 1], dp0[i + 1][j - 1]}) + m[i][j];
        }

        // step 4 : merge the three states
        for (int i = 1; i <= n; i++) {

```

```

        int j = antidiag - i;
        if (j < 1 || j > n) continue;
        dp[i][j] = max({dp0[i][j], dp1[i][j], dp2[i][j]});
    }
}

fout << dp[n][n] << "\n";
return 0;
}

```

The program follows precisely the recurrences presented in the editorial, uses the same antidiagonal ordering, and outputs the required maximum sum.